



Guida tecnica del Cruscotto Degenze

Com'è fatto, come funziona, come si modifica
e come si porta sulla macchina virtuale

Casa di Cura Misericordia Navacchio
Aggiornata al 21/09/2026

Indice

1. Il programma in breve
2. Come funziona: browser, server e 127.0.0.1
3. Le tecnologie usate
4. Le cartelle del progetto
5. Il database
6. Accesso, ruoli e sicurezza
7. Mettere mano al programma
8. L'installer e la prova sul PC dell'infermiere
9. Fase 2: la macchina virtuale
10. Comandi utili
11. Glossario

Come leggere questa guida

I capitoli 1-4 spiegano l'idea generale e bastano per capire che cosa succede quando qualcuno apre il Cruscotto. I capitoli 5-7 servono quando si vuole modificare qualcosa. I capitoli 8 e 9 riguardano l'installazione: oggi sul PC di reparto, domani sulla macchina virtuale. Le parole tecniche meno comuni sono spiegate nel glossario in fondo.

1. Il programma in breve

Il **Cruscotto Degenze** è un'applicazione web interna della Casa di Cura Misericordia Navacchio. Serve a vedere in un colpo d'occhio i pazienti ricoverati nei tre reparti (AOUP, Setting 1 — Low Care 1, Setting 2 — Low Care 2), a organizzare le dimissioni e a lasciare note fra un turno e l'altro.

Cosa si può fare:

- vedere per ogni reparto i pazienti ricoverati, con letto, date e un pallino colorato di gravità (non valutato, stabile, instabile, critico);
- inserire, spostare di letto o reparto e dimettere un paziente;
- scrivere, modificare ed eliminare note nell'**Agenda** del paziente; ogni modifica conserva il testo precedente;
- spuntare le 21 voci della **checklist di dimissione**;
- scaricare la **scheda del paziente in PDF**, con dati, checklist e note;
- gestire gli utenti (solo l'amministratore): crearli, cambiarne il ruolo, reimpostare la password, disattivarli, sbloccarli.

Cosa il Cruscotto non è

Non è una cartella clinica: i dati clinici restano in ResMedica. Qui si registra solo il minimo necessario a organizzare degenza e dimissione.

Le due fasi del progetto

	Fase 1 — oggi	Fase 2 — domani
Dove gira	Sul singolo PC, installato con il setup	Su una macchina virtuale nel datacenter
Chi lo usa	Solo chi siede a quel PC	Tutti i PC di reparto del dominio
Indirizzo	http://127.0.0.1:8000	https://degenze.cdcmisnav.local
Database	SQLite: il file cruscotto.db	SQL Server su sql-misnav
Server	Waitress in una finestra nera	Waitress come servizio Windows, dietro IIS
Cifratura	Non serve: il traffico non esce dal PC	HTTPS con il certificato della CA interna

Il codice è **lo stesso** nelle due fasi. Cambiano solo il file di configurazione `.env` e ciò che sta intorno al programma (IIS, servizio, database).

2. Come funziona: browser, server e 127.0.0.1

Un'applicazione web è divisa in due. Il **browser** (Edge, Chrome) mostra le pagine e raccoglie i clic. Il **server** è un programma sempre in ascolto che riceve le richieste del browser, legge o scrive i dati e risponde con una pagina. Nel Cruscotto il server è la famosa «finestra nera»: finché resta aperta il programma funziona, chiudendola si spegne.

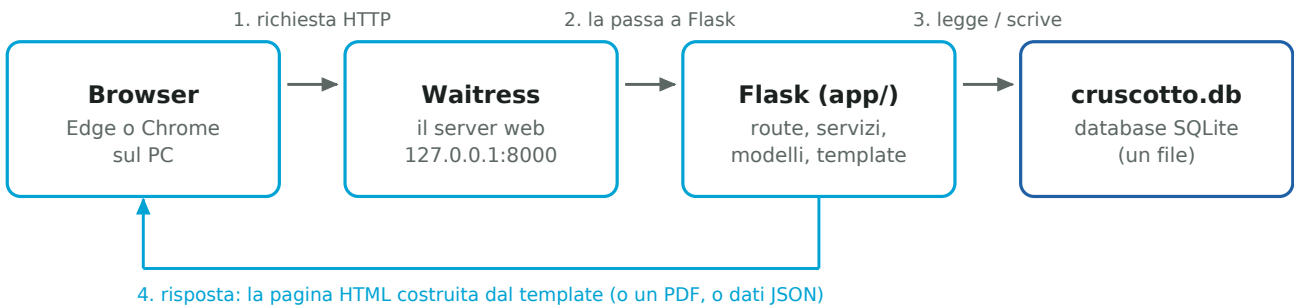


Figura 1 — Il percorso di una richiesta nella Fase 1, tutto dentro lo stesso PC.

Che cos'è 127.0.0.1

Ogni computer in rete ha un indirizzo IP, per esempio 192.168.0.45. Oltre a quello, ogni computer ha un indirizzo speciale che significa sempre «**me stesso**»: **127.0.0.1**, chiamato anche *localhost* o indirizzo di *loopback*. Il traffico verso 127.0.0.1 non esce mai dalla scheda di rete: va e torna dentro lo stesso computer.

Il server del Cruscotto si mette in ascolto proprio su 127.0.0.1. Conseguenza pratica: **solo il PC su cui è installato può aprirlo**. Un altro PC che scrivesse 127.0.0.1 nel browser parlerebbe con se stesso, dove il Cruscotto non c'è. È una protezione voluta, ed è il motivo per cui la Fase 1 non ha bisogno di HTTPS: i dati non viaggiano sulla rete.

Le porte: :8000 e :5000

Su un computer possono girare molti server insieme. Per distinguerli, ognuno ascolta su una **porta** diversa, un numero da 1 a 65535. L'indirizzo completo è IP più porta:

Indirizzo	Chi risponde	Quando si usa
http://127.0.0.1:8000	Waitress, il server «vero»	Programma installato con il setup
http://127.0.0.1:5000	Il server di sviluppo di Flask (flask run)	Mentre si modifica il codice
https://degenze.cdcmisnav.local	IIS sulla VM (porta 443, sottintesa)	Fase 2

Usare due porte diverse permette di tenere acceso il programma installato e, contemporaneamente, la copia su cui si sta lavorando, senza che si disturbino.

E se volessi aprirlo da un altro PC?

Bisognerebbe far ascoltare il server su 0.0.0.0 (cioè su tutte le schede di rete) e aprire la porta nel firewall di Windows. Il traffico però viaggierebbe in chiaro, password comprese: si fa solo con dati finti. La strada giusta per l'uso condiviso è la Fase 2 (capitolo 9).

Cosa succede quando si clicca «Accedi»

1. Il browser invia al server nome utente e password (una richiesta *POST* all'indirizzo `/accedi`).
2. Flask cerca la funzione associata a quell'indirizzo, la **route** `login()` in `app/auth/routes.py`.
3. La route chiama i **servizi** (`app/auth/servizi.py`), che contengono la logica: l'utente esiste? è attivo? è bloccato? la password è giusta?
4. I servizi usano i **modelli** (`app/models.py`) per leggere la tabella utente e aggiornare il contatore dei tentativi falliti.
5. Se va tutto bene, Flask crea la **sessione**: un cookie firmato con la `SECRET_KEY`, che il browser rimanderà a ogni richiesta successiva per dire «sono sempre io».
6. Il browser viene rimandato alla home; la route della home legge i pazienti e li passa al **template** `pazienti/home.html`, che costruisce la pagina HTML.

Tutto il programma segue questo schema: **route** → **servizi** → **modelli** → **template**. Quando si cerca dove intervenire, basta chiedersi a quale dei quattro livelli appartiene la modifica.

3. Le tecnologie usate

Tutte gratuite e open source, tutte installate dal setup o da `pip install -r requirements.txt`.

Componente	A cosa serve
Python 3.12	Il linguaggio in cui è scritto il programma.
Flask	Il «telaio» dell'applicazione web: collega gli indirizzi (route) alle funzioni Python.
Jinja2	Il motore dei template: pagine HTML con dei «buchi» che Flask riempie con i dati.
SQLAlchemy + Flask-SQLAlchemy	L'ORM: permette di usare il database con classi Python invece di scrivere SQL a mano. Lo stesso codice funziona con SQLite e con SQL Server.
Alembic + Flask-Migrate	Le <i>migrazioni</i> : modifiche al database scritte come file, applicate in ordine.
Flask-Login	Gestisce chi è collegato e protegge le pagine riservate.
Flask-WTF	I form, con il token anti-CSRF.
ReportLab	Genera i PDF (scheda paziente, informativa, questa guida).
Waitress	Il server web di produzione che fa girare Flask.
python-dotenv	Legge le impostazioni dal file <code>.env</code> .
pytest	Esegue i test automatici (circa 200).
Inno Setup	Costruisce il setup <code>.exe</code> (solo sul PC di sviluppo).

Niente internet

Il programma non carica nulla da internet: niente font di Google, niente librerie da CDN. CSS e JavaScript sono scritti a mano in `app/static`. Così funziona anche con la rete esterna staccata. Lo garantisce anche la Content-Security-Policy (capitolo 6): se qualcuno aggiungesse per sbaglio un link esterno, il browser lo bloccherebbe subito.

4. Le cartelle del progetto

Cruscotto-Degenze\ app\ __init__.py config.py models.py dati_fissi.py permessi.py audit.py sessione.py comandi.py formati.py auth\ pazienti\ agenda\ pdf\ utenti\ templates\ static\ migrations\versions\ tests\ docs\ Installer\ PromptIA\ wsgi.py requirements.txt .env cruscotto.db	IL PROGRAMMA create_app(): costruisce l'applicazione impostazioni Sviluppo / Test / Produzione le tabelle del database, come classi Python reparti e voci della checklist chi puo' fare cosa (matrice dei ruoli) registro delle attivita' scadenza per inattivita', cambio password obbligato comandi da terminale: crea-admin, dati-demo, ... date in italiano, fuso orario accesso, uscita, cambio password, profilo home, nuovo paziente, gravita', sposta, dimetti note e checklist scheda paziente e informativa in PDF gestione utenti (solo amministratore) le pagine HTML (Jinja2) css\stile.css, js\app.js, immagini, font le migrazioni del database, una per file i test automatici diario di sviluppo e questa guida costruzione del setup .exe specifica iniziale e prototipo grafico punto di ingresso per Waitress librerie Python, con versione bloccata impostazioni locali e segreti (NON va su Git) il database SQLite (NON va su Git)
--	--

Ogni area funzionale (auth, pazienti, agenda, pdf, utenti) è un **blueprint** di Flask, cioè un pezzo di programma con le sue route. Dentro ciascuna si ritrovano gli stessi file:

File	Contiene
routes.py	Le funzioni collegate agli indirizzi: ricevono la richiesta, controllano i permessi, chiamano i servizi, scelgono il template.
servizi.py	La logica vera: regole, controlli, letture e scritture sul database. Non sanno nulla di HTML.
form.py	I campi dei moduli e i loro controlli (obbligatorio, lunghezza massima, ...).
__init__.py	Crea il blueprint.

Perché separare route e servizi

I servizi si possono provare con i test senza passare dal browser, e la stessa regola (per esempio «un letto non può essere occupato da due pazienti») sta in un posto solo, anche se la usano più pagine.

5. Il database

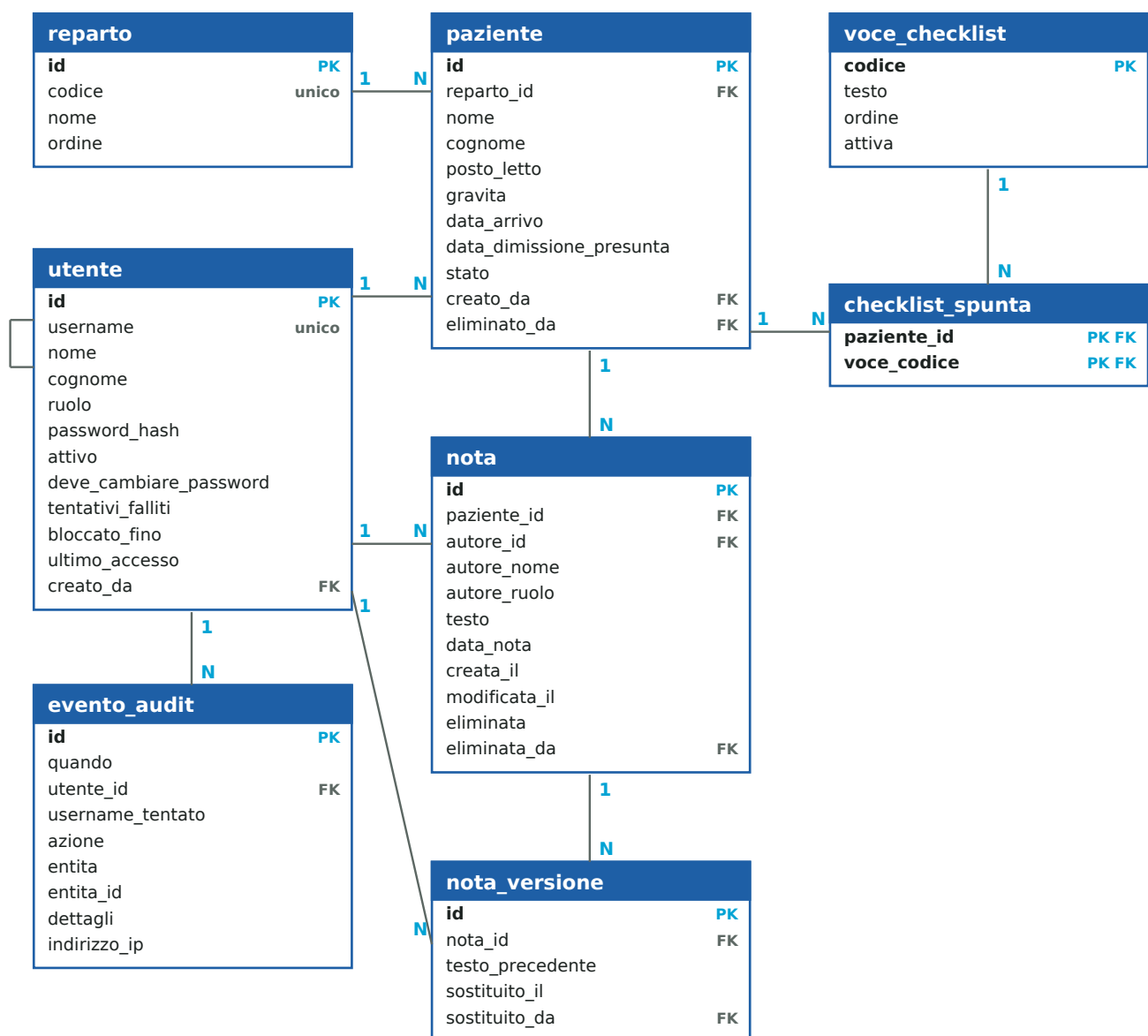
5.1 SQLite ora, SQL Server dopo

Nella Fase 1 il database è **SQLite**: un unico file, `cruscotto.db`, nella cartella del programma. Non c'è un server di database da installare: la libreria che lo legge è già dentro Python. Nella Fase 2 lo stesso schema verrà creato su **SQL Server**.

Il codice non scrive quasi mai SQL a mano: usa **SQLAlchemy**, che traduce le operazioni sulle classi di `models.py` nel dialetto SQL giusto per il database in uso. Per passare da SQLite a SQL Server basta cambiare la riga `DATABASE_URL` nel file `.env`.

5.2 Lo schema ER

Lo schema **entità-relazioni** (ER) mostra le tabelle, le colonne più importanti e i collegamenti. Una linea «1 — N» vuol dire che a una riga della prima tabella corrispondono molte righe della seconda: un reparto ha molti pazienti, un paziente ha molte note, e così via.



PK chiave primaria: identifica la riga
FK chiave esterna: punta a un'altra tabella
1 — N una riga di qua, molte di là

Figura 2 — Schema ER. Le tabelle sono otto. Il piccolo anello a sinistra di «utente» indica che creato_da punta a un altro utente: chi ha creato l'account.

checklist_spunta è una **tabella ponte**: realizza la relazione «molti a molti» fra pazienti e voci della checklist (un paziente ha molte voci spuntate, una voce è spuntata per molti pazienti). La sua chiave primaria è la coppia paziente + voce.

5.3 Le tabelle una per una

Tabella	Cosa contiene	Da sapere
reparto	I tre reparti.	Dati fissi, inseriti dalla migrazione iniziale (app/dati_fissi.py).

Tabella	Cosa contiene	Da sapere
utente	Gli account del personale.	La password non c'è: c'è solo il suo hash. Ruoli: ADMIN, MEDICO, INFERMIERE, OSS, AMMIN.
paziente	I pazienti, ricoverati e dimessi.	stato = RICOVERATO o ELIMINATO. gravita = nv, verde, giallo, rosso.
nota	Le note dell'Agenda.	Nome e ruolo dell'autore sono copiati al momento della scrittura: la firma resta giusta anche se poi l'utente cambia ruolo.
nota_versione	Il testo precedente di ogni nota modificata.	Una riga per ogni modifica: la storia completa si ricostruisce sempre.
voce_checklist	Le 21 voci della checklist.	Chiave = codice (es. PRIVACY). attiva = false nasconde una voce senza perderne lo storico.
checklist_spunta	Quali voci sono spuntate per quale paziente.	La riga esiste = voce spuntata. È l'unica tabella in cui si cancella davvero.
evento_audit	Il registro delle attività.	Chi, quando, cosa, da quale IP. Si scrive e basta: non si modifica mai.

5.4 Le regole importanti

- **Non si cancella quasi nulla.** Un paziente dimesso non sparisce: passa a stato ELIMINATO, con data e autore. Una nota eliminata resta con `eliminata = true`. Così si può sempre ricostruire cosa è successo. (Nella Fase 2 lo garantirà anche SQL Server, negando all'utente del programma il permesso DELETE.)
- **Un letto, un paziente.** L'indice `ux_paziente_letto_occupato` impedisce due pazienti RICOVERATI nello stesso letto dello stesso reparto. È un indice *parziale*: i pazienti dimessi non contano, altrimenti un letto usato una volta resterebbe bloccato per sempre.
- **La dimissione non precede l'arrivo.** Un vincolo CHECK nel database lo impedisce anche se l'applicazione avesse un bug.
- **Orari in UTC.** Date e ore di sistema (`creata_il`, `quando`, ...) sono salvate in tempo universale e convertite in ora italiana solo per la visualizzazione (`app/formati.py`). Così il cambio dell'ora legale non crea buchi o doppioni.
- **Registro delle attività.** Ogni operazione importante scrive una riga in `evento_audit`: accessi riusciti e falliti, uscite, pazienti creati, spostati, dimessi, note, PDF scaricati, gestione utenti. La pagina per consultarlo è ancora da fare (passo 8b).

5.5 Guardare dentro il database

Per curiosare nel file `cruscotto.db` il modo più comodo è **DB Browser for SQLite** (gratuito, sqlitebrowser.org). Apri il file, scheda *Sfogliati dati*, scegli la tabella.

Regole di prudenza

Apri il database in **sola lettura** (c'è l'opzione nel menu Apri). Non modificare i dati a mano mentre il programma è acceso. Per fare una copia di sicurezza, spegni il programma (chiudi la finestra nera) e copia il file `cruscotto.db` altrove: è tutto lì dentro.

5.6 Le migrazioni

Una **migrazione** è un file Python che descrive una modifica al database: crea una tabella, aggiunge una colonna, inserisce una voce. Stanno in `migrations/versions/`, ciascuna sa qual è la precedente e il database ricorda (nella tabella `alembic_version`) a che punto è arrivato. Il comando `flask db upgrade` applica in ordine quelle che mancano: per questo un aggiornamento non perde i dati.

Migrazione	Cosa fa
042c1d4c264a	Struttura iniziale: le otto tabelle, i reparti e le voci della checklist.
b7e2a91c4d10	Aggiunge la voce «Modulo Privacy Somministrato» ai database già esistenti.

```
# dopo aver modificato app/models.py:
flask db migrate -m "descrizione breve" # scrive la migrazione
# RILEGGI il file creato in migrations\versions\ prima di andare avanti
flask db upgrade                        # la applica al tuo database
flask db downgrade                      # torna indietro di una
```

La regola d'oro

Una migrazione già applicata su un altro PC (o sulla VM) **non si modifica mai**: se c'è da correggere qualcosa, se ne scrive una nuova. Il PC dell'infermiere non rieseguirebbe la migrazione cambiata, e i due database prenderebbero strade diverse.

6. Accesso, ruoli e sicurezza

6.1 Ruoli e permessi

La matrice sta in un solo file, `app/permessi.py`. Ogni route protetta ha il decoratore `@richiede_permesso`, che risponde «403 — non consentito» a chi non ha diritto. Nascondere un pulsante nella pagina serve solo a non confondere: il controllo vero è sempre sul server.

Permesso	Ammin.	Medico	Infermiere	OSS	Ammin./IT
Vedere pazienti, Agenda, checklist	✓	✓	✓	✓	✓
Scrivere note, spuntare la checklist	✓	✓	✓	—	—
Cambiare la gravità	✓	✓	✓	—	—
Inserire, spostare, dimettere pazienti	✓	✓	✓	—	—
Modificare la dimissione presunta	✓	✓	✓	—	—
Scaricare la scheda PDF	✓	✓	✓	—	✓
Gestire gli utenti	✓	—	—	—	—

6.2 Le protezioni

Misura	Come funziona	Dove
Password protette	Salvate come hash script: non si possono rileggere, solo verificare.	<code>models.py</code>
Cambio al primo accesso	Chi riceve una password dall'amministratore deve sceglierne una propria.	<code>sessione.py</code>
Blocco dopo 5 tentativi	Dopo 5 password sbagliate l'account resta bloccato 15 minuti. L'amministratore può sbloccarlo prima.	<code>auth/servizi.py</code> , <code>config.py</code>
Scadenza per inattività	Dopo 15 minuti senza attività la sessione si chiude: i PC di reparto sono condivisi.	<code>sessione.py</code> , <code>config.py</code>
Token CSRF	Ogni modulo contiene un codice segreto: un sito esterno non può inviare moduli al posto dell'utente.	Flask-WTF
Content-Security-Policy	Il browser esegue solo script e stili serviti dal Cruscotto stesso.	<code>app/__init__.py</code>
Cookie protetti	Il cookie di sessione non è leggibile dagli script; in produzione viaggia solo su HTTPS.	<code>config.py</code>
Registro attività	Tutto ciò che conta finisce in <code>evento_audit</code> .	<code>audit.py</code>

6.3 Il file `.env`

Contiene le impostazioni che cambiano da un'installazione all'altra e i segreti. Non va mai su Git (lo esclude `.gitignore`) e non va mandato per email.

```
SECRET_KEY=...lunga stringa casuale... # firma i cookie di sessione
FLASK_CONFIG=sviluppo                  # sviluppo | test | produzione
DATABASE_URL=                           # vuoto = cruscotto.db (SQLite)
MINUTI_INATTIVITA=15
FLASK_APP=wsgi.py
```

Se si cambia la `SECRET_KEY`, tutti gli utenti collegati devono rifare l'accesso. Il setup ne genera una casuale e non la riscrive negli aggiornamenti.

7. Mettere mano al programma

7.1 Preparare il PC di sviluppo

Servono Python 3.12, Git e Visual Studio Code. Poi, una volta sola, dalla cartella del progetto:

```
py -3.12 -m venv .venv          # crea l'ambiente virtuale
.\.venv\Scripts\Activate.ps1    # lo attiva (a ogni nuovo terminale)
pip install -r requirements.txt  # installa le librerie
copy .env.example .env          # poi metti una SECRET_KEY nel .env
flask db upgrade                 # crea cruscotto.db
flask crea-admin                 # il tuo amministratore
flask dati-demo                  # facoltativo: pazienti e utenti finti
```

Cos'è l'ambiente virtuale (.venv)

Una cartella con una copia privata di Python e delle librerie del progetto, nelle versioni esatte di requirements.txt. Così aggiornare una libreria per un altro programma non rompe il Cruscotto. Non va su Git: si ricrea con i comandi qui sopra.

7.2 Il ciclo di lavoro

1. Avvia il server di sviluppo: `flask run --debug` e apri `http://127.0.0.1:5000`. Con `--debug` il server si riavvia da solo a ogni modifica: basta ricaricare la pagina (Ctrl+F5 se è cambiato CSS o JavaScript).
2. Fai la modifica.
3. Lancia i test: `pytest`. Devono passare tutti. Se ne fallisce uno, leggi il messaggio: spesso è il test a dirti cosa hai rotto.
4. Salva il lavoro su Git: `git add .` e `git commit -m "cosa ho fatto"`.
5. Se la modifica deve arrivare al PC dell'infermiere, ricompila il setup (capitolo 8) e reinstallalo sulla stessa cartella.

7.3 Dove si tocca per...

Voglio...	File da toccare
Cambiare un testo in una pagina	app/templates/... (cerca la frase con Ctrl+Shift+F in VS Code)
Cambiare colori, spaziature, caratteri	app/static/css/stile.css (i colori principali sono variabili in cima al file)
Cambiare un comportamento nella pagina (finestre, clic)	app/static/js/app.js
Aggiungere o togliere una voce della checklist	app/dati_fissi.py + una nuova migrazione (vedi 7.4)
Aggiungere un reparto	app/dati_fissi.py + una nuova migrazione, come per la checklist
Cambiare chi può fare cosa	app/permessi.py (e i test in tests/)
Aggiungere un campo al paziente	models.py, poi flask db migrate; il form in pazienti/form.py; la pagina in templates/pazienti; il PDF in pdf/generatore.py
Cambiare i minuti di blocco o di inattività	config.py o, meglio, il file .env
Cambiare il PDF del paziente	app/pdf/generatore.py
Cambiare l'informativa legale	app/pdf/informativa.py

Voglio...	File da toccare
Cambiare l'installer	Installer/CruscottoDegenze.iss e Installer/script/

7.4 Esempio completo: una voce nuova nella checklist

Così è stata aggiunta «Modulo Privacy Somministrato»:

1. In `app/dati_fissi.py` si aggiunge in fondo a `VOCI_CHECKLIST` la riga (`"PRIVACY", "Modulo Privacy Somministrato"`). Il codice (`PRIVACY`) è la chiave: massimo 20 caratteri, maiuscolo, senza spazi.
2. Si scrive una migrazione che inserisce la voce nei database già esistenti (`migrations/versions/b7e2a91c4d10_...py`). Inserisce solo se la voce manca, perché sui database nuovi la mette già la migrazione iniziale.
3. `flask db upgrade` sul proprio database e `flask controlla-dati-fissi` per verificare: «Voci della checklist: 21 (attese 21)».
4. `pytest`: i test che contavano le voci ora le leggono dall'elenco invece di avere il numero scritto a mano.
5. Commit, ricompilazione del setup, reinstallazione: il setup esegue `flask db upgrade` da solo, e la voce compare senza toccare pazienti e spunte.

7.5 Lavorare con l'assistente IA

- Chiedi una modifica alla volta, piccola e ben descritta.
- Fatti spiegare cosa ha cambiato e in quali file; rileggi le differenze in VS Code (scheda Controllo del codice sorgente) prima del commit.
- Pretendi che i test passino, e che per ogni comportamento nuovo ci sia un test.
- Mai incollare nella chat password vere, il contenuto del `.env` o dati di pazienti reali.

8. L'installer e la prova sul PC dell'infermiere

8.1 Costruire il setup

Il setup CruscottoDegenze-Setup.exe contiene una **copia** dei file del programma: dopo ogni modifica va ricompilato, altrimenti installa la versione vecchia. Dalla cartella del progetto:

```
powershell -ExecutionPolicy Bypass -File Installer\Compila.ps1
```

Il risultato è Installer\Output\CruscottoDegenze-Setup.exe (circa 4 MB). Serve Inno Setup 6: se manca, lo script lo installa con winget. Il .exe non va su Git (è in .gitignore): se si scarica il progetto da capo, va ricompilato.

8.2 Cosa fa il setup

#	Passo
1	Chiede la cartella e i dati del primo amministratore (nome, cognome, nome utente, password).
2	Copia i file in %LOCALAPPDATA%\Programs\CruscottoDegenze. Non in C:\Programmi, dove il programma non potrebbe scrivere il proprio database senza permessi di amministratore.
3	Installa Python 3.12, se non c'è.
4	Crea l'ambiente virtuale .venv e scarica le librerie (serve internet).
5	Scrivi il .env con una SECRET_KEY casuale.
6	Crea o aggiorna il database con flask db upgrade.
7	Crea l'amministratore e mette i collegamenti su Desktop e menu Start.

8.3 La prova sul PC dell'infermiere, passo per passo

1. Copia CruscottoDegenze-Setup.exe sulla chiavetta.
2. Sul PC dell'infermiere, collegato a internet, fai doppio clic sul file. Se compare «Windows ha protetto il PC», clicca *Ulteriori informazioni* e poi *Esegui comunque*: succede a ogni programma senza firma digitale.
3. Segui la procedura e inserisci i dati del primo amministratore (tu). L'installazione non chiede i permessi di amministratore di Windows.
4. Alla fine si apre la finestra nera e poi il browser su <http://127.0.0.1:8000>. Entra con l'amministratore.
5. Da *Gestione utenti* crea l'account dell'infermiere, con ruolo Infermiere e una password provvisoria. Al primo accesso gli verrà chiesto di cambiarla.
6. Spiega che la **finestra nera è il programma**: va lasciata aperta (si può ridurre a icona). Chiudendola il Cruscotto si spegne.

Da sapere per la prova

I dati inseriti restano **solo su quel PC**, nel file cruscotto.db della cartella di installazione, e nessun altro PC li vede. Non c'è un backup automatico. Per la prova è meglio usare dati finti o ridotti al minimo; se si usano dati reali, il PC deve essere protetto dalla password di Windows e l'informativa (link sulla pagina di accesso) va condivisa con chi partecipa.

8.4 Aggiornare, salvare, disinstallare

- **Aggiornare:** chiudi la finestra nera e rilancia il nuovo setup sulla stessa cartella. Database, utenti e .env restano; la pagina del primo amministratore viene saltata; il database viene portato all'ultima versione.
- **Copia di sicurezza:** a programma spento, copia %LOCALAPPDATA%\Programs\CruscottoDegenze\cruscotto.db su una chiavetta o in una cartella sicura. Per ripristinarlo, rimetti il file al suo posto.
- **Disinstallare:** da Impostazioni di Windows → App, come ogni programma.

8.5 Problemi frequenti

Sintomo	Rimedio
Il setup si ferma sulle librerie	Manca internet. Collega il PC e rilancia solo la configurazione: powershell -ExecutionPolicy Bypass -File installazione\Configura.ps1 dalla cartella del programma.
«Il Cruscotto è acceso»	Chiudi la finestra nera e riprova.
Il browser non si apre	Il server è partito più lentamente: vai a mano su http://127.0.0.1:8000 .
Utente bloccato	Aspetta 15 minuti, oppure l'amministratore lo sblocca da Gestione utenti.
Password dell'amministratore dimenticata	Dalla cartella del programma: .venv\Scripts\flask.exe crea-admin, per creare un secondo amministratore.

9. Fase 2: la macchina virtuale

Nella Fase 2 il Cruscotto diventa un servizio di rete: un'unica installazione su una macchina virtuale del datacenter, raggiunta da tutti i PC di reparto all'indirizzo **https://degenze.cdcmisnav.local**, con i dati su SQL Server e i backup del datacenter.

9.1 L'architettura

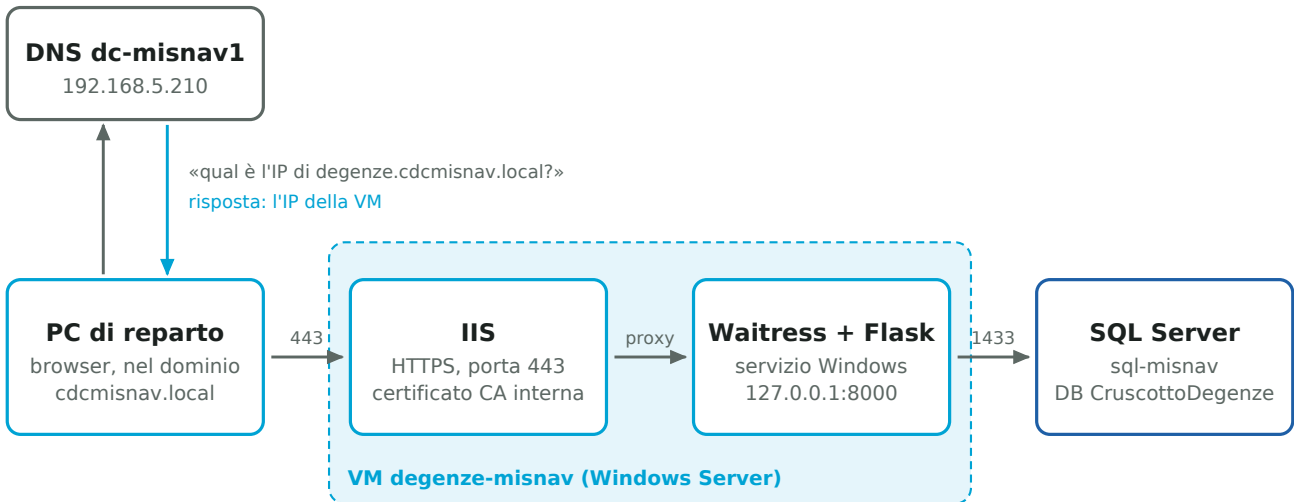


Figura 3 — Il percorso di una richiesta nella Fase 2.

1. Il PC chiede al DNS (dc-misnav1) l'indirizzo di degenze.cdcmisnav.local e riceve l'IP della VM.
2. Il browser si collega in HTTPS sulla porta 443. Il traffico passa dai FortiGate, che devono permetterlo.
3. Sulla VM risponde **IIS**, con il certificato della CA interna ca-misnav: i PC del dominio lo riconoscono, quindi niente avvisi.
4. IIS fa da **reverse proxy**: non esegue il programma, ma inoltra la richiesta a Waitress, che ascolta solo su 127.0.0.1:8000 e quindi è irraggiungibile da fuori della VM.
5. Waitress fa girare Flask, che legge e scrive su SQL Server (sql-misnav, porta 1433).

Perché IIS davanti a Waitress

Waitress è ottimo per le applicazioni Python ma non gestisce HTTPS. IIS sì, usa i certificati della CA interna ed è già conosciuto in struttura. Ognuno fa ciò che sa fare meglio: IIS cifra, Waitress esegue il programma.

9.2 Cosa manca ancora nel codice

Il programma è stato scritto pensando alla Fase 2 (SQLAlchemy, configurazione «produzione», indici compatibili con SQL Server), ma alcune cose vanno ancora preparate **prima** dell'installazione. La documentazione iniziale le cita come se esistessero: ad oggi **non ci sono**.

Da fare	Perché
Cartella deploy/ con installa.ps1, aggiorna.ps1, crea_database.sql, web.config e la guida DEPLOY.md	Per installare e aggiornare la VM in modo ripetibile, senza affidarsi alla memoria.

Da fare	Perché
Attivare ProxyFix in wsgi.py	Dietro IIS ogni richiesta arriva da 127.0.0.1: senza ProxyFix il registro delle attività scriverebbe sempre quell'IP invece di quello del PC di reparto.
Scommentare pyodbc in requirements.txt e installare l'ODBC Driver 18 sulla VM	È il «ponte» fra Python e SQL Server.
Log su file (cartella logs/)	Un servizio non ha una finestra: gli errori devono finire in un file da consultare.
Pagina di consultazione del registro attività (passo 8b)	Gli eventi sono già scritti, manca la pagina per leggerli.
Prova completa delle migrazioni su SQL Server	Da fare nella prova generale (9.3): SQLite perdona cose che SQL Server non perdona.

Per esempio, ProxyFix sono due righe in wsgi.py:

```
from werkzeug.middleware.proxy_fix import ProxyFix
app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1)
```

9.3 Prima: la prova generale sul tuo PC

Prima di toccare il datacenter conviene ripetere tutta l'installazione su una VM di prova. Se la prova va liscia, l'installazione vera sarà una ripetizione.

- **Hyper-V:** su Windows 11 Pro si attiva da PowerShell come amministratore con `Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Hyper-V -All`, poi si riavvia.
- **Sistema:** Windows Server 2025 Evaluation, gratuito per 180 giorni.
- **Database:** SQL Server Express, gratuito, sulla stessa VM.
- **Risorse:** 2 vCPU, 4 GB di RAM, 60 GB di disco (sul PC servono almeno 16 GB di RAM).
- **Certificato:** autofirmato; il browser mostrerà un avviso, ed è normale.
- Prova anche un **backup e un ripristino** del database: un backup mai provato non è un backup.

9.4 Cosa chiedere al consulente (Marco Bottalico)

- **VM degenze-misnav:** Windows Server 2025 Standard, 2 vCPU, 4 GB RAM, 80 GB di disco, nel dominio cdcmisnav.local, IP statico nella rete 192.168.5.0/24, agente SentinelOne.
- **SQL Server su sql-misnav:** conferma che può ospitare il database CruscottoDegenze, nome esatto dell'istanza, autenticazione mista attiva, database incluso nel piano di backup.
- **Regole FortiGate:** postazioni 192.168.0.0/24 verso la VM sulle porte 443 e 80; VM verso sql-misnav sulla porta 1433.
- **Certificato:** modello «Server Web» pubblicato sulla CA ca-misnav, con permesso di richiesta per la VM.

9.5 I passi di installazione

1. **DNS:** su dc-misnav1, un record A «degenze» nella zona cdcmisnav.local che punta all'IP della VM.
2. **Database e login su sql-misnav:** un database CruscottoDegenze e due login, uno per il programma (lettura e scrittura) e uno solo per le migrazioni (proprietario).

3. **Prerequisiti sulla VM:** Python 3.12 per tutti gli utenti, ODBC Driver 18 for SQL Server, ruolo IIS, moduli URL Rewrite e Application Request Routing, NSSM.
4. **Programma:** copia in C:\Apps\CruscottoDegenze, crea .env, installa le librerie, scrivi il .env di produzione (9.6), lancia flask db upgrade con il login delle migrazioni.
5. **Servizio Windows** con NSSM, perché Waitress parta da solo all'accensione:

```
nssm install CruscottoDegenze C:\Apps\CruscottoDegenze\.env\Scripts\waitress-serve.exe
nssm set CruscottoDegenze AppParameters "--listen=127.0.0.1:8000 wsgi:app"
nssm set CruscottoDegenze AppDirectory C:\Apps\CruscottoDegenze
nssm start CruscottoDegenze
```

6. **Certificato:** da certlm.msc sulla VM, Personale → Richiedi nuovo certificato, modello «Server Web», con degenze.cdcmisnav.local nel campo SAN.
7. **IIS:** sito sulla porta 443 con il certificato; in Application Request Routing → Server Proxy Settings abilita il proxy; una regola che inoltra tutto a http://127.0.0.1:8000 e una che porta la porta 80 su HTTPS.
8. **Firewall di Windows:** in ingresso solo le porte 80 e 443.
9. **Verifica:** da un PC di reparto apri https://degenze.cdcmisnav.local. Deve comparire il lucchetto senza avvisi; login, note, checklist e PDF devono funzionare. Riavvia la VM e controlla che il servizio riparta da solo.
10. **Utenti reali:** flask crea-admin per il primo amministratore, poi dal programma gli account di medici e infermieri.

Uno scheletro del web.config di IIS (da provare nella prova generale):

```
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="Da HTTP a HTTPS" stopProcessing="true">
          <match url="(.*)" />
          <conditions><add input="{HTTPS}" pattern="off" /></conditions>
          <action type="Redirect" url="https://{HTTP_HOST}/{R:1}" />
        </rule>
        <rule name="Cruscotto" stopProcessing="true">
          <match url="(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:8000/{R:1}" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

9.6 Il .env di produzione

```
FLASK_APP=wsgi.py
FLASK_CONFIG=produzione
SECRET_KEY=<nuova stringa casuale, diversa da quella dei PC>
DATABASE_URL=mssql+pyodbc://degenze_app:<password>@sql-misnav.cdcmisnav.local/CruscottoDegenze?driver=ODBC+Driver+for+SQL+Server
MINUTI_INATTIVITA=15
```

Con FLASK_CONFIG=produzione il programma si rifiuta di partire senza SECRET_KEY, spegne la modalità debug e manda il cookie di sessione solo su HTTPS. DATABASE_URL va scritta su una sola riga.

Due punti da verificare con il consulente

Certificato di SQL Server: se non è emesso dalla CA interna, la connessione con Encrypt=yes può fallire; temporaneamente si aggiunge TrustServerCertificate=yes, poi si sistema il certificato.

PC nel vecchio dominio misericordianavacchio.local: potrebbero non risolvere cdcmisnav.local e non fidarsi della CA ca-misnav. Servono un inoltro condizionale DNS e il certificato della CA distribuito via GPO.

9.7 Portare i dati dalla Fase 1 alla Fase 2

I dati della prova sui PC (SQLite) **non passano** da soli su SQL Server. La via consigliata è partire puliti sulla VM: la prova serve a raccogliere commenti, non a produrre dati da conservare. Se servisse davvero travasarli, si può scrivere uno script apposito, da provare prima sulla VM di prova.

9.8 Backup, aggiornamenti, controlli

- **Backup del database:** completo e giornaliero su sql-misnav, con conservazione da concordare (per esempio 30 giorni). È la cosa più importante: il programma si reinstalla in mezz'ora, i dati no.
- **VM:** snapshot prima di ogni aggiornamento. **.env:** copia in un luogo sicuro ad accesso limitato, perché contiene le password.
- **Aggiornare il programma,** fuori dall'orario di punta: backup del database → arresto del servizio → nuovo codice → pip install -r requirements.txt → flask db upgrade → riavvio del servizio → verifica. Se qualcosa va storto si ripristinano codice e database salvati.
- **Controlli periodici:** log degli errori; account di chi lascia la struttura disattivati subito; una volta al mese verifica degli account attivi; aggiornamenti di Windows e, passando prima dalla VM di prova, delle librerie Python.

10. Comandi utili

Da un terminale nella cartella del progetto, con l'ambiente virtuale attivo (.\.venv\Scripts\Activate.ps1):

```
flask run --debug                # server di sviluppo su http://127.0.0.1:5000
waitress-serve --listen=127.0.0.1:8000 wsgi:app  # come il programma installato

pytest                          # tutti i test
pytest -v                       # elencandoli uno per uno
pytest tests\test_pdf.py        # solo un file

flask db upgrade                 # porta il database all'ultima versione
flask db migrate -m "...       # nuova migrazione dopo aver cambiato models.py
flask db downgrade              # annulla l'ultima migrazione
flask db current                 # a che migrazione e' il database

flask crea-admin                # crea un amministratore
flask dati-demo                 # dati di prova (solo in sviluppo)
flask dati-demo --azzera        # come sopra, ripartendo da zero
flask controlla-dati-fissi      # reparti e voci della checklist ci sono tutti?

python docs\genera_guida.py     # rigenera questa guida
python docs\genera_diario.py    # rigenera il diario di sviluppo

powershell -ExecutionPolicy Bypass -File Installer\Compila.ps1 # ricompila il setup

git status                      # cosa e' cambiato
git add .                      # prepara tutto per il commit
git commit -m "messaggio"      # salva una versione
git log --oneline               # la storia delle versioni
```

11. Glossario

Termine	Significato
127.0.0.1 / localhost	L'indirizzo che per ogni computer significa «me stesso». Il traffico non esce dal PC.
Porta	Numero che distingue i server in ascolto sullo stesso computer (8000, 5000, 443...).
HTTP / HTTPS	Il protocollo con cui browser e server si parlano. HTTPS è la versione cifrata, con certificato.
Route	Associazione fra un indirizzo (/accedi) e la funzione Python che risponde.
Template	Pagina HTML con dei segnaposto che Flask riempie con i dati (Jinja2).
Blueprint	Un pezzo del programma Flask con le sue route (auth, pazienti, agenda, pdf, utenti).
ORM	Libreria (SQLAlchemy) che fa usare il database come oggetti Python, senza scrivere SQL.
Migrazione	File che descrive una modifica al database, applicato con flask db upgrade.
Chiave primaria / esterna	La primaria identifica una riga; l'esterna punta a una riga di un'altra tabella.
Hash	Trasformazione a senso unico: dalla password si ottiene l'hash, ma dall'hash non si torna alla password.
Sessione / cookie	Il «tesserino» firmato che il browser mostra a ogni richiesta per dire chi è.
CSRF	Attacco in cui un sito esterno invia moduli a nome dell'utente. Il token CSRF lo impedisce.
Ambiente virtuale	Cartella .venv con Python e le librerie del progetto, separati dal resto del PC.
Reverse proxy	Server (IIS) che riceve le richieste dall'esterno e le inoltra al programma vero (Waitress).
Servizio Windows	Programma che parte da solo all'accensione e gira senza finestra (con NSSM).
DNS	Il «servizio elenco» che traduce un nome (degenze.cdcmisnav.local) in un indirizzo IP.
Certificato / CA	Documento digitale che prova l'identità del server; la CA interna (ca-misnav) lo firma.
Audit	Registro delle attività: chi ha fatto cosa, quando, da dove.